Evo at Tool Competition – UAV Testing Track

Mattia Davoli
Politecnico di Milano
Milano, Italy
mattia.davoli@mail.polimi.it

Matteo Camilli
Politecnico di Milano
Milano, Italy
matteo.camilli@polimi.it

***Abstract*—Evo is a test-case generation tool that looks for vulnerabilities in the PX4 obstacle-avoidance system within the Aerialist test bench. It is built on a *multi-seed* Evolutionary Algorithm that evolves several seeds independently. Evo reconstructs the UAV mission trajectory and combines *trajectory-aware anchor seeding* with a *geometric pre-filter* that discards obstacles too far from the planned path.**

***Index Terms*—Tool Competition, Software Testing, Search-Based Techniques, Evolutionary Algorithms, Unmanned Aerial Vehicles.**

I. Introduction

Unmanned Aerial Vehicles (UAVs) are increasingly deployed in commercial domains such as agriculture, surveillance, and logistics, where safety and reliability are critical. As UAVs become more autonomous, rigorous and automated testing of their safety-critical components becomes essential. To encourage research in this direction, a UAV Testing Tool Competition is held annually [4].

Aerialist (unmanned AERIAL vehIcle teST bench) is a UAV test bench, built on top of the PX4 firmware [5], that automates the definition, generation, execution, and analysis of system-level UAV test cases in simulation [2], [3]. A test case is described by properties of the drone (software configuration, autopilot parameters, mission plan), of the environment (simulated physics, virtual world, initial position, and the placement and dimensions of obstacles), and by the commands issued during the mission.

This paper describes **Evo**, a test generator that works with Aerialist to expose weaknesses of the PX4 obstacle-avoidance system. Evo adopts a search-based approach: it manipulates the size, position, and orientation of a small number of obstacles with the goal of either causing the drone to crash or significantly diverting it from its intended path. Evo evolves several seeds independently, each anchored to a different portion of the planned trajectory, in order to discover *diverse* as well as *effective* failure scenarios within a fixed simulation budget.

II. Evo

Evo employs the $(1{+}1)$ Evolutionary Strategy (ES) [1]. The search starts from a set of *seeds*, i.e. initial obstacle configurations generated randomly and kept only when they lie within the planned trajectory; among the valid candidates, the most promising one is selected as the starting point. Each seed is evolved independently: it acts as a single parent that is iteratively refined through mutation, and a mutated configuration replaces the current one whenever it improves the fitness. Using several seeds spread along the planned path mitigates premature convergence to a single local optimum and naturally produces spatially diverse tests.

A. Parameters and Hyperparameters

A test case in Evo is fully described by a small set of obstacles, and the search acts directly on their geometry. We separate the quantities that the evolutionary process adjusts at run time, the *parameters*, from those that remain fixed throughout a run and shape the search behaviour, the *hyperparameters*.

Every obstacle contributes five parameters: its planar position (x, y), its orientation (r), and its two horizontal dimensions (length l, width w). All of them are subject to mutation and may take any value within the bounds defined by the competition [4]. The remaining geometric quantities are kept fixed: the height h is set to its maximum value ($25\,\mathrm{m}$), the value suggested for the competition, so that obstacles always reach the flight altitude, while the vertical coordinate z stays at ground level. Neither affects the outcome, so excluding them from the search concentrates the budget on the parameters that matter.

The hyperparameters are chosen once, before the search starts. They include the number of seeds from which the evolution begins, the number of obstacles per test case, the standard deviation of the mutation, and the distance thresholds used to keep candidate obstacles close to the trajectory. Among these, the number of obstacles deserves attention: although up to three are permitted, Evo deliberately favours configurations with two. Since the per-test score scales with the inverse square of the obstacle count, a failure found with two obstacles is worth considerably more than the same failure found with three.

B. Initialization

Evo begins by reconstructing the UAV's flight path from the mission plan. The waypoints $\mathcal{M} = \{W_1, W_2, \ldots, W_n\}$ are read from the `.plan` file and converted from geographic to local metric coordinates, forming the planned path P.

a) Anchor seeding: The path P is partitioned by arc length into equal portions, and the midpoint of each portion is used as an *anchor* a_k. Evo creates one seed per anchor (four by default). When a seed is built, its first obstacle is sampled within a disk of radius ρ around the corresponding anchor, while the remaining obstacle(s) are placed randomly along the trajectory. In all cases, the geometric pre-filter ensures that every obstacle lies close to the planned path.

b) Geometric pre-filter: Every candidate configuration must be geometrically valid: obstacles may not overlap, and *each* obstacle must lie within the pre-filter threshold θ_{pre} of the path. Configurations that do not satisfy these constraints are rejected before any simulation is spent on them.

c) Seed selection: For each anchor, Evo simulates up to MAX_INIT valid candidates and immediately adopts the first one that is *challenging*, i.e. whose minimum drone-to-obstacle distance $d_{\min}$ is below d_{crit}. If no candidate turns out to be challenging, the seed is initialized with the most promising one tried so far, i.e. the configuration that brought the drone closest to an obstacle. In the non-anchored configuration, a global pool of discarded candidates provides a fallback. Each selected seed is the parent from which the remaining budget is then spent on mutation.

C. Algorithm

Each seed is evolved independently with the $(1{+}1)$ ES, starting from its own parent configuration. At every iteration a mutation perturbs a single randomly chosen obstacle and a single aspect of it – its *position*, its *size*, or its *rotation* – by adding Gaussian noise whose standard deviation is a fixed fraction σ of the corresponding range, clipped to the admissible bounds. The mutated child must remain geometrically valid.

The child is then simulated and evaluated. It is accepted as the new parent only if its fitness is strictly greater than the parent's. Every evaluated configuration whose minimum drone-to-obstacle distance satisfies $d_{\min} < d_{\text{crit}}$ is recorded in the archive as a critical scenario, regardless of whether it is retained as a parent.

The fitness function is a continuous approximation of the competition score and is defined over two regimes. In the challenging region, where $d_{\min}$ is below the critical threshold, the fitness grows as the drone passes closer to an obstacle. Outside this region, where the configuration is still safe, the fitness is negative and increases as the drone approaches the threshold, providing a smooth gradient that pulls a harmless configuration toward a challenging one even before any failure has been observed.

Once all seeds have exhausted their budget, Evo post-processes the archive. Test cases are first *de-duplicated* using an area-based similarity between the obstacle footprints: two tests whose similarity is at least τ are considered duplicates, and only one of them is kept. The surviving tests are then sorted by $(d_{\min}, \#\text{obst})$ in ascending order, placing the most dangerous and lowest-obstacle tests first, and the top-ranked tests are emitted as the final suite.

Algorithm 1 Evo

Require: mission plan, simulation budget B
Ensure: ranked archive of critical test cases

$P \leftarrow \textsc{Trajectory}(\text{plan})$
$\{a_1, \ldots, a_K\} \leftarrow \textsc{SegmentMidpoints}(P)$
$\mathcal{A} \leftarrow \emptyset$ ▷ archive
for each anchor a_k with budget b **do**
  $x_{\text{cur}} \leftarrow \textsc{InitSeed}(a_k)$ ▷ anchor seeding + pre-filter
  evaluate x_{cur}; **if** critical **then** add to $\mathcal{A}$
  while budget not exhausted **do**
    $x_{\text{new}} \leftarrow \textsc{Mutate}(x_{\text{cur}})$ ▷ pos / size / rot
    if x_{new} not valid **then**
      continue
    end if
    evaluate fitness of x_{new}
    if $\text{fit}(x_{\text{new}}) > \text{fit}(x_{\text{cur}})$ **then**
      $x_{\text{cur}} \leftarrow x_{\text{new}}$
    end if
    if $d_{\min}(x_{\text{new}}) < d_{\text{crit}}$ **then**
      add x_{new} to $\mathcal{A}$
    end if
  end while
end for
$\mathcal{A} \leftarrow \textsc{Deduplicate}(\mathcal{A}, \tau)$ ▷ Jaccard/IoU
sort $\mathcal{A}$ by $(d_{\min}, \#\text{obst})$ ascending
return top tests of $\mathcal{A}$

The general procedure is summarized above in Algorithm 1.

III. Conclusion

Evo demonstrates that combining a multi-seed $(1{+}1)$ Evolutionary Strategy with trajectory-aware anchor seeding and a competition-aligned fitness function is an effective way to discover diverse and dangerous failure scenarios for the PX4 obstacle-avoidance system under a tight simulation budget.

References

[1] J. Campos, Y. Ge, N. Albunian, G. Fraser, M. Eler, and A. Arcuri, "An empirical evaluation of evolutionary algorithms for unit test suite generation," *Information and Software Technology*, vol. 104, pp. 207–235, 2018.

[2] S. Khatiri, S. Panichella, and P. Tonella, "Simulation-based test case generation for unmanned aerial vehicles in the neighborhood of real flights," in *2023 IEEE Int. Conf. on Software Testing, Verification and Validation (ICST)*, 2023.

[3] S. Khatiri, S. Panichella, and P. Tonella, "Simulation-based testing of unmanned aerial vehicles with Aerialist," in *Int. Conf. on Software Engineering (ICSE)*, 2024.

[4] S. Khatiri, T. Zohdinasab, P. Saurabh, D. Humeniuk, and S. Panichella, "ICST tool competition 2025 – UAV testing track," in *IEEE/ACM Int. Conf. on Software Testing, Verification and Validation (ICST)*, 2025.

[5] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *Int. Conf. on Robotics and Automation (ICRA)*, pp. 6235–6240, IEEE, 2015.